



University of Asia Pacific
Department of Computer Science and
Engineering

Compiler Design
(CSE 430)

Submitted By

Mazharul Islam Sourav

ID: 22101133

Section: C-2

Semester: 4-2

Submitted To

Md Shafayatul Haque

Lecturer

Department of Computer Science
and Engineering

University of Asia Pacific

Work 01: Write a program for lexical analysis.

Code -

```
class Information:
    def __init__(self, name, type_val, size, dimension, line_of_code,
address):
        self.name = name
        self.type = type_val
        self.size = size
        self.dimension = dimension
        self.line_of_code = line_of_code
        self.address = address
        self.next = None

class Table:
    def __init__(self, max_size=10):
        self.MAX = max_size
        self.table = [None] * self.MAX

    def Hashindex(self, name):
        hash_value = 0
        for char in name:
            hash_value += ord(char)
        return hash_value % self.MAX

    def insert(self, name, type_val, size, dimension, line_of_code,
address):

        if self.search(name):
            print(f"Symbol '{name}' already exists in the symbol table!")
            return False

        new_symbol = Information(name, type_val, size, dimension,
line_of_code, address)

        index = self.Hashindex(name)
```

```

        if self.table[index] is None:
            self.table[index] = new_symbol
        else:
            new_symbol.next = self.table[index]
            self.table[index] = new_symbol

    print(f"Inserted '{name}' at index {index}")
    return True

def search(self, name):
    index = self.Hashindex(name)
    current = self.table[index]

    while current is not None:
        if current.name == name:
            return current
        current = current.next
    return None

def delete(self, name):
    index = self.Hashindex(name)
    current = self.table[index]
    prev = None

    while current is not None:
        if current.name == name:
            if prev is None:
                self.table[index] = current.next
            else:
                prev.next = current.next
            print(f"Deleted '{name}' from index {index}")
            return True
        prev = current
        current = current.next

    print(f"Symbol '{name}' not found!")
    return False

```

```

def update(self, name, new_name=None, type_val=None, size=None,
dimension=None, line_of_code=None, address=None):
    symbol = self.search(name)

    if symbol is None:
        print(f"Symbol '{name}' not found!")
        return False

    if new_name is not None and self.search(new_name):
        print(f"Symbol '{new_name}' already exists! Cannot rename.")
        return False

    if new_name is not None:
        old_index = self.Hashindex(name)
        new_index = self.Hashindex(new_name)

        current = self.table[old_index]
        prev = None
        while current is not None:
            if current.name == name:
                if prev is None:
                    self.table[old_index] = current.next
                else:
                    prev.next = current.next
                break
            prev = current
            current = current.next

        symbol.name = new_name
        symbol.next = None
        if self.table[new_index] is None:
            self.table[new_index] = symbol
        else:
            symbol.next = self.table[new_index]
            self.table[new_index] = symbol

    if type_val is not None:
        symbol.type = type_val
    if size is not None:
        symbol.size = size

```

```

        if dimension is not None:
            symbol.dimension = dimension
        if line_of_code is not None:
            symbol.line_of_code = line_of_code
        if address is not None:
            symbol.address = address

        print(f"Updated '{name}'" + (f" -> '{new_name}'" if new_name else
""))

        return True

    def show(self):

print(f"\n{'Index':<8}{'Name':<12}{'Type':<12}{'Size':<8}{'Dimension':<12}
{'Line':<8}{'Address'}")
        print("-" * 68)
        empty = True
        for i in range(self.MAX):
            current = self.table[i]
            while current is not None:
                empty = False

print(f"{i:<8}{current.name:<12}{current.type:<12}{current.size:<8}{current
t.dimension:<12}{current.line_of_code:<8}{current.address}")
                current = current.next

        if empty:
            print("(Empty table)")
        print()

def main():
    symbol_table = Table(max_size=10)

    while True:
        print("\nInsert")
        print("Search")
        print("Delete")
        print("Update")
        print("Show")
        print("Exit")

```

```

choice = input("\nEnter your choice: ").upper()

if choice == 'INSERT':
    entry = input("Enter (name type size dimension line address): ")

    parts = entry.split()

    if len(parts) != 6:
        print("Invalid input! Please enter exactly 6 values separated by spaces.")
    else:
        symbol_table.insert(parts[0], parts[1], parts[2], parts[3], parts[4], parts[5])

elif choice == 'SEARCH':
    name = input("Enter name to search: ")
    result = symbol_table.search(name)
    if result:
        print(f"\nFound '{name}':")
        print(f"  Type: {result.type}")
        print(f"  Size: {result.size}")
        print(f"  Dimension: {result.dimension}")
        print(f"  Line: {result.line_of_code}")
        print(f"  Address: {result.address}")
    else:
        print(f"Symbol '{name}' not found!")

elif choice == 'DELETE':
    name = input("Enter name to delete: ")
    symbol_table.delete(name)

elif choice == 'UPDATE':
    name = input("Enter name to update: ")
    entry = input("Enter new values (name type size dimension line address): ")
    parts = entry.split()

    if len(parts) != 6:

```

```
        print("Invalid input! Please enter exactly 6 values
separated by spaces.")
    else:
        new_name = parts[0] if parts[0] != '-' else None
        type_val = parts[1] if parts[1] != '-' else None
        size = parts[2] if parts[2] != '-' else None
        dimension = parts[3] if parts[3] != '-' else None
        line_of_code = parts[4] if parts[4] != '-' else None
        address = parts[5] if parts[5] != '-' else None
        symbol_table.update(name, new_name, type_val, size,
dimension, line_of_code, address)

    elif choice == 'SHOW':
        symbol_table.show()

    elif choice == 'EXIT':
        print("Chole Jaccho.....Abar esho !!")
        break
    else:
        print("Hoy nai !! Abar try koro !!")

if __name__ == "__main__":
    main()
```

Output -

```
Insert
Search
Delete
Update
Show
Exit

Enter your choice: insert
Enter (name type size dimension line address): faisal human 85kg 3d 103 #gjasdj
Inserted 'faisal' at index 4
```

```
Insert
Search
Delete
Update
Show
Exit
```

Enter your choice: show

Index	Name	Type	Size	Dimension	Line	Address
4	faisal	human	85kg	3d	103	#gjasdj

```
Insert
Search
Delete
Update
Show
Exit
```

```
Insert
Search
Delete
Update
Show
Exit
```

Enter your choice: search
Enter name to search: faisal

Found 'faisal':
Type: human
Size: 85kg
Dimension: 3d
Line: 103
Address: #gjasdj

Enter your choice: show

Index	Name	Type	Size	Dimension	Line	Address
1	emon	human	75kg	2d	203	#gdsa
4	faisal	human	85kg	3d	103	#gjajsdj

Insert
Search
Delete
Update
Show
Exit

Enter your choice: delete
Enter name to delete: emon
Deleted 'emon' from index 1

Insert
Search
Delete
Update
Show
Exit

Enter your choice: show

Index	Name	Type	Size	Dimension	Line	Address
4	faisal	human	85kg	3d	103	#gjajsdj

Enter your choice: update
Enter name to update: faisal
Enter new values (name type size dimension line address): z z z z z z
Updated 'faisal' -> 'z'

Insert
Search
Delete
Update
Show
Exit

Enter your choice: show

Index	Name	Type	Size	Dimension	Line	Address
2	z	z	z	z	z	z

Insert
Search
Delete
Update
Show
Exit

Code Link : [GitHub](#)